Java 17, IntelliJ nowy UI podobny do VS

Otwórz projekt – **spring-boot-02**. Na wejściu dodajmy kilka klas które będziemy rozbudowywali.

Mamy moduł vod-app

W projekcie będziemy się zajmowali kinami i filmami, model będzie prosty: kino, film, reżyser.

Mamy pakiet model a w nim 3 klasy: Cinema, Director, Movie

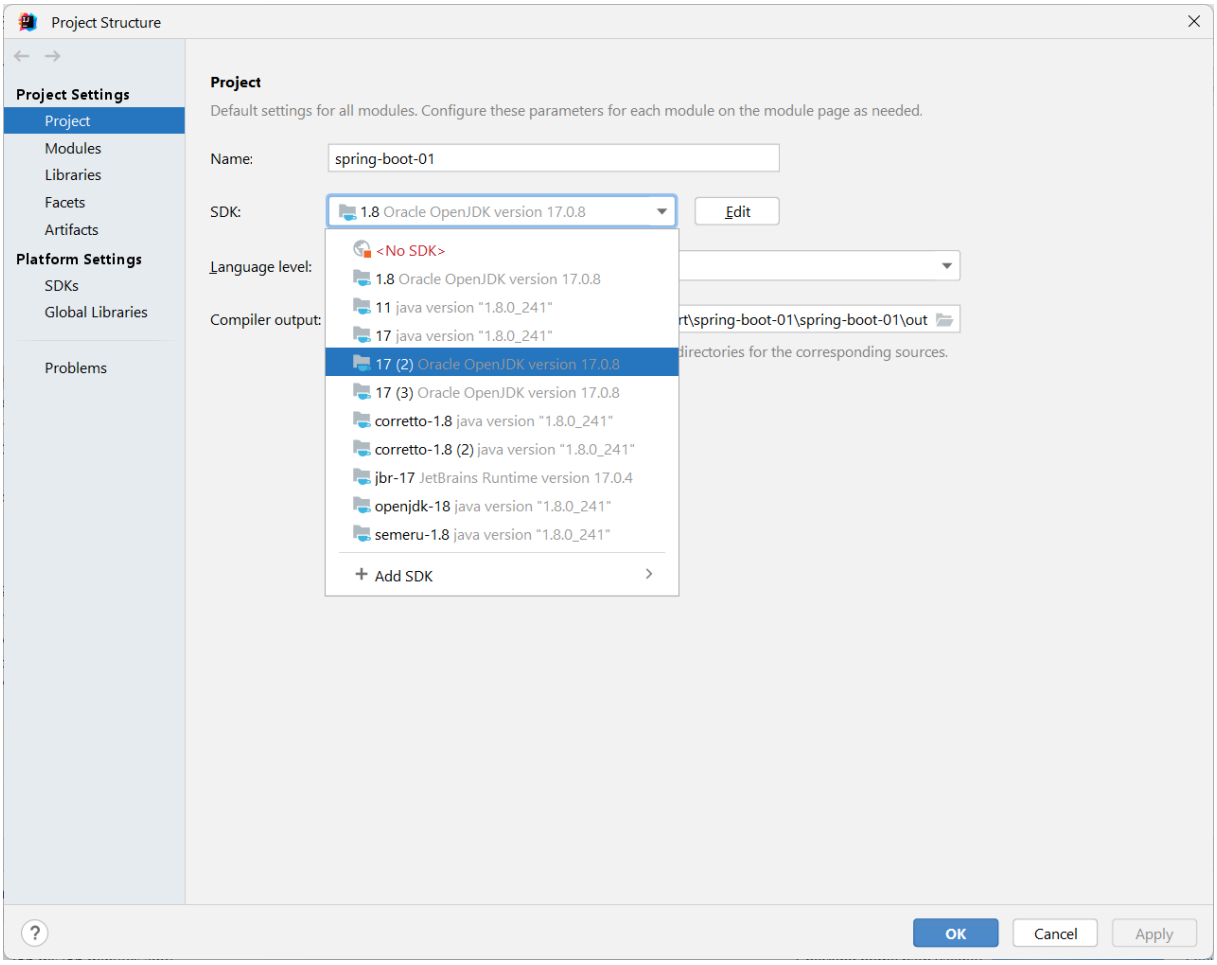
Mamy do tego 3 repozytoria zdefiniowane jako interfejsy w pakiecie repository

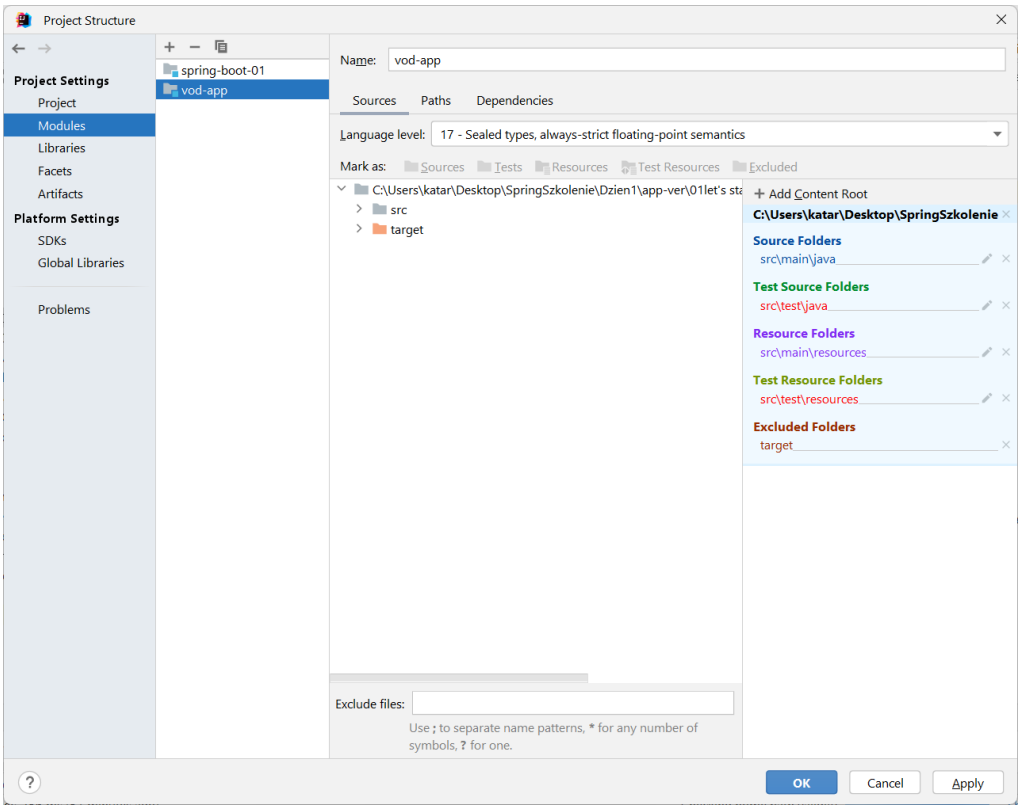
Oraz inicjalnie implementację w pamięci - memory

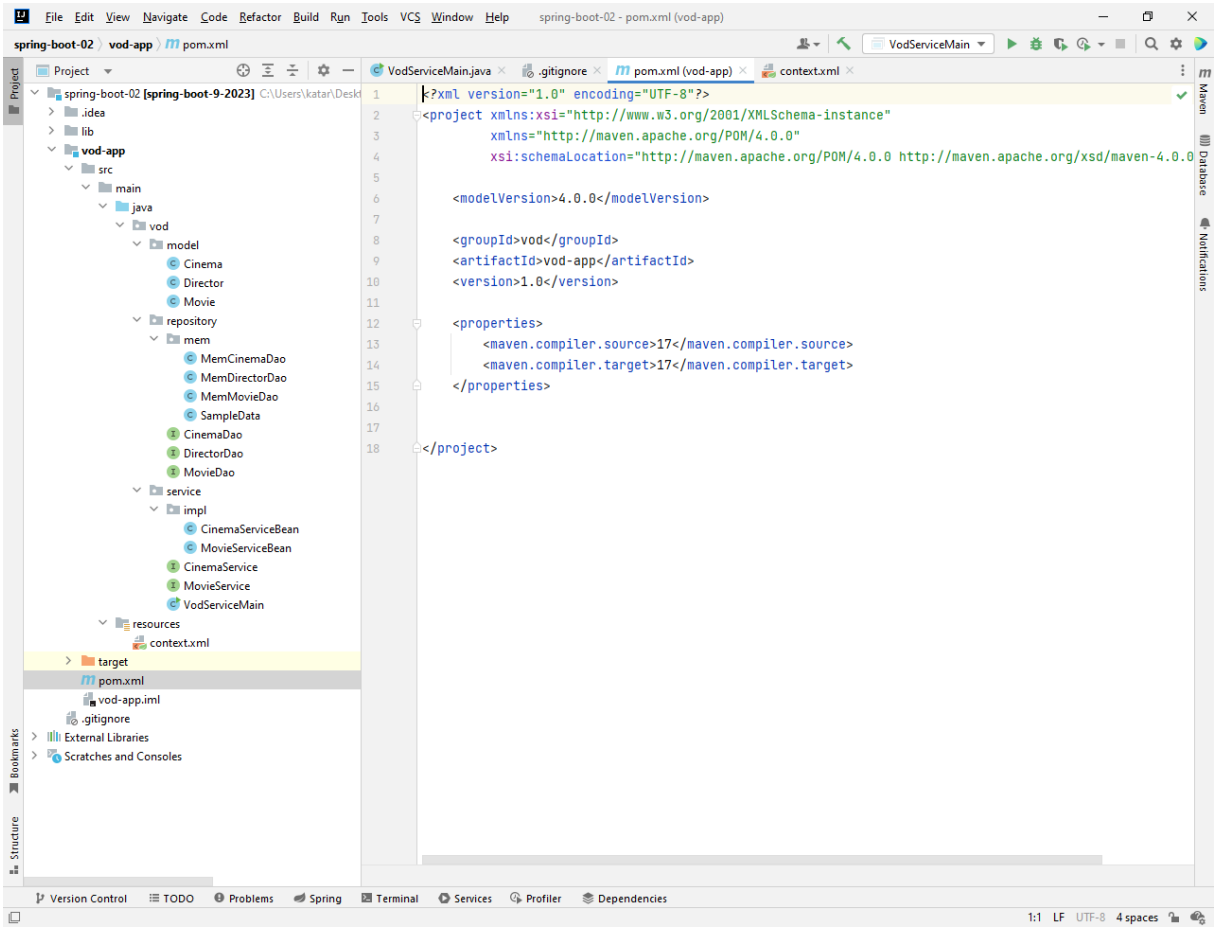
Jest tu też warstwa usług – service, gdzie zdefiniowane są dwa serwisy

Jest to projekt mavenowy (add as Maven Project). Ściąganie zależności może trochę potrwać. Zwróć uwagę na ikonkę pliku pom.xml

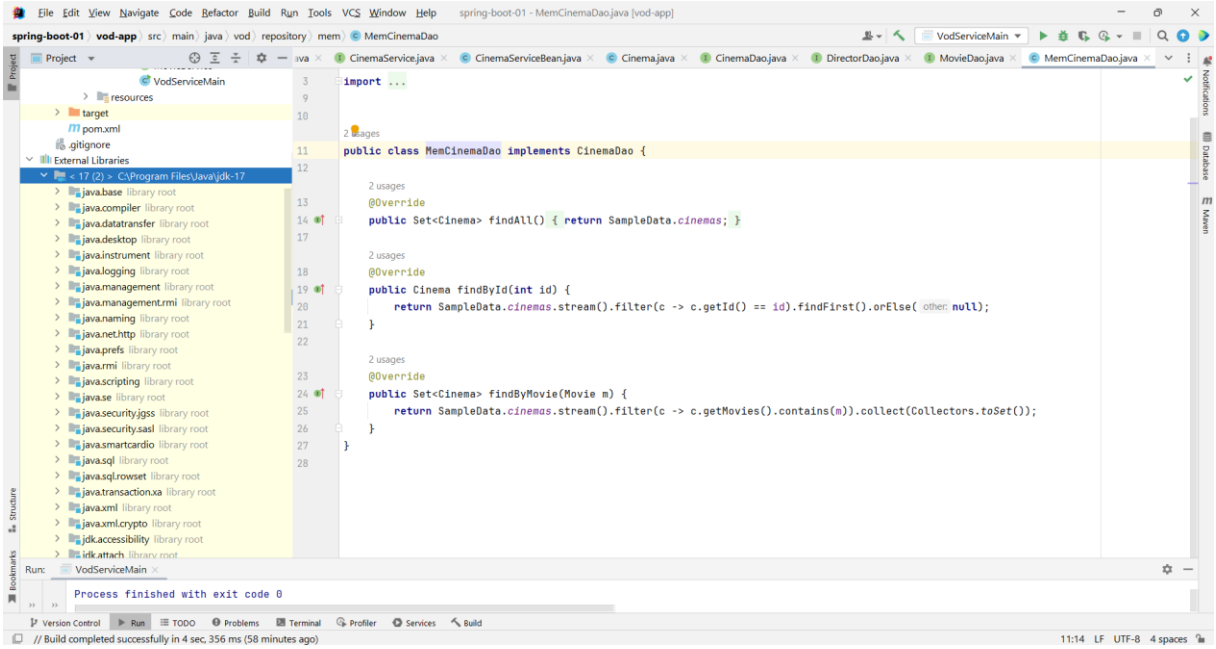
Podłącz JDK – 17.




Project Structure
Project Settings
Project
Modules
Libraries
Facets
Artifacts
Platform Settings
SDKs
Global Libraries
Problems
spring-boot-01
vod-app
Name: vod-app
Sources
Paths
Dependencies
Language level: 17 - Sealed types, always-strict floating-point semantics
Mark as: Sources Tests Resources Test Resources Excluded
C:\Users\katar\Desktop\SpringSzkolenie\Dzien1\app-ver\01let's sta
src
target
Add Content Root
C:\Users\katar\Desktop\SpringSzkolenie
Source Folders
src\main\java
Test Source Folders
src\test\java
Resource Folders
src\main\resources
Test Resource Folders
src\test\resources
Excluded Folders
target
Exclude files:
Use ; to separate name patterns, * for any number of symbols, ? for one.
OK
Cancel
Apply


File Edit View Navigate Code Refactor Build Run Tools VCS Window Help
spring-boot-02 - pom.xml (vod-app)
spring-boot-02 vod-app pom.xml
VodServiceMain
Project
spring-boot-02 [spring-boot-9-2023]
.idea
lib
vod-app
src
main
java
vod
model
Cinema
Director
Movie
repository
mem
MemCinemaDao
MemDirectorDao
MemMovieDao
SampleData
CinemaDao
DirectorDao
MovieDao
service
impl
CinemaServiceBean
MovieServiceBean
CinemaService
MovieService
VodServiceMain
resources
context.xml
target
pom.xml
vod-app.iml
.gitignore
External Libraries
Scratches and Consoles
VodServiceMain.java
.gitignore
pom.xml (vod-app)
context.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://maven.apache.org/POM/4.0.0"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0
<modelVersion>4.0.0</modelVersion>
<groupId>vod</groupId>
<artifactId>vod-app</artifactId>
<version>1.0</version>
<properties>
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
</properties>
</project>
Version Control
TODO
Problems
Spring
Terminal
Services
Profiler
Dependencies
1:1 LF UTF-8 4 spaces

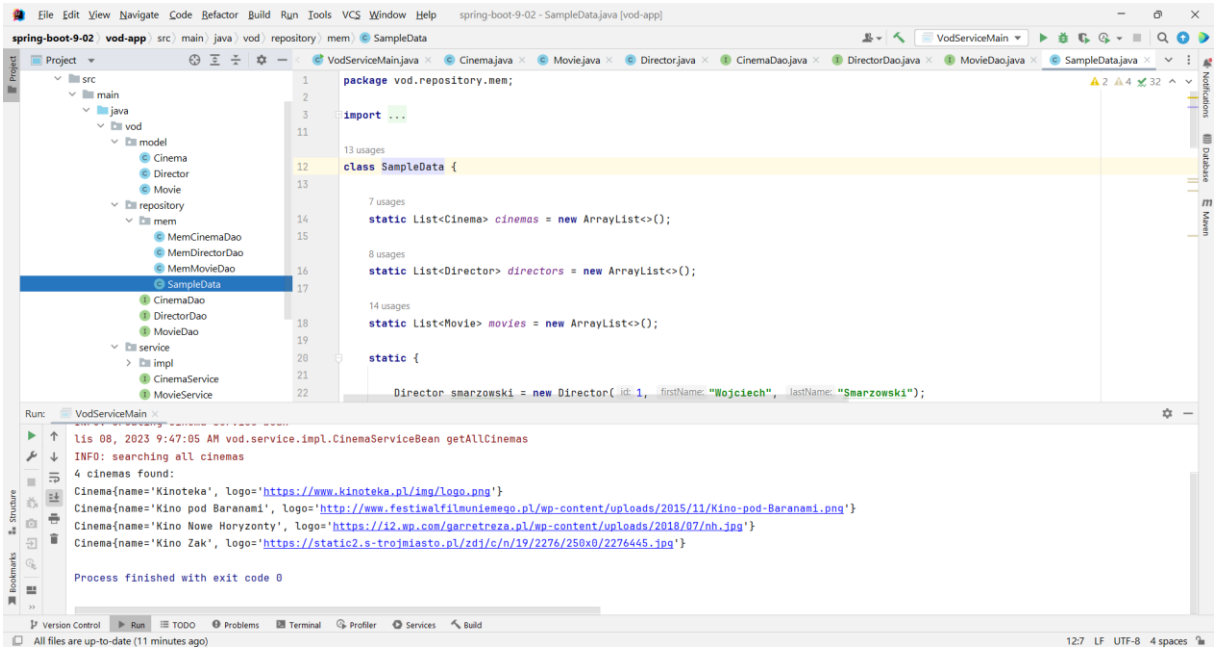
Uwaga: na zrzutach widzimy Sety ale w danych mamy po refaktoryzacji Listy bo ich będziemy używać.



W projekcie mamy bardzo prosty **model danych** definiujący koncepcję kina i filmu, relacja wiele do wiele – bi directional.

Omówienie modelu.

Wokół modelu danych zbudowane są **repozytoria – czyli obiekty DAO,** reprezentowane poprzez interfejsy, które implementują zarządzanie danymi i reprezentują dostęp do danych do middleware systemu zewnętrznego (metody find, add), na początku będą one w pamięci zafixowane – klasa **SampleData**, później w bazie danych. Kilka filmów, reżyserów, kin. To chwilowe początkowe rozwiązanie źródła danych (zaślepka). W oparciu o te dane działają obiekty klas MemCinemaDao, MemDirectorDao i MemMovieDao. Z punktu widzenia interfejsów nie wiemy jaka implementacja ich jest użyta, jaki rodzaj bazy.



Mamy obiekty danych i **repozytoria** - które symbolizują zarządzanie tymi obiektami i są reprezentowane poprzez interfejsy, więc możemy korzystać z dowolnej implementacji.

W następnej kolejności wzorcowo - warstwa usług - zgodnie z programowaniem **DDD** (Domain Driven Design). Jest to sposób tworzenia aplikacji i systemów, które w głównej mierze powinny w jak najlepszy sposób odzwierciedlać potrzeby biznesu – a więc założenia biznesowe, które stają się jednocześnie warunkami koniecznymi zawartymi w oprogramowaniu. Domain Driven Design to metoda tworzenia oprogramowania, która jest wykorzystywana w największych biznesowych systemach informatycznych.

Dziedzina domenowa to warstwa dziedziny / usług tzw. **serwisy** - jest to zaimplementowana **logika biznesowa** naszej aplikacji, której implementacja serwisu będzie korzystała z obiektów DAO (repo) w postaci interfejsów.

Serwisy nie będą wiązały się bezpośrednio z implementacją samą w sobie ale będą oparte na typach interfejsowych (repozytoriów), które są również podane jako parametry w konstruktorze. Na poziomie importów serwisów nie mamy nic o technologii, jest przygotowany na to że middleware może się zmienić poprzez implementacje różnych interfejsów repo. Javowy fragment kodu abstrahujący od technologii realizujący logikę biznesową i jest przygotowany na to, że middleware może się zmienić poprzez różne implementacje interfejsów.

```
package vod.service.impl;

import vod.repository.CinemaDao;
import vod.repository.MovieDao;
import vod.model.Cinema;
import vod.model.Movie;
import vod.service.CinemaService;

import java.util.Set;
import java.util.logging.Logger;

2 usages
public class CinemaServiceBean implements CinemaService {

    5 usages
    private static final Logger log = Logger.getLogger(CinemaService.class.getName());

    4 usages
    private CinemaDao cinemaDao;
    2 usages
    private MovieDao movieDao;

    1 usage
    public CinemaServiceBean(CinemaDao cinemaDao, MovieDao movieDao) {
        log.info( msg: "creating cinema service bean");
        this.cinemaDao = cinemaDao;
        this.movieDao = movieDao;
    }
```

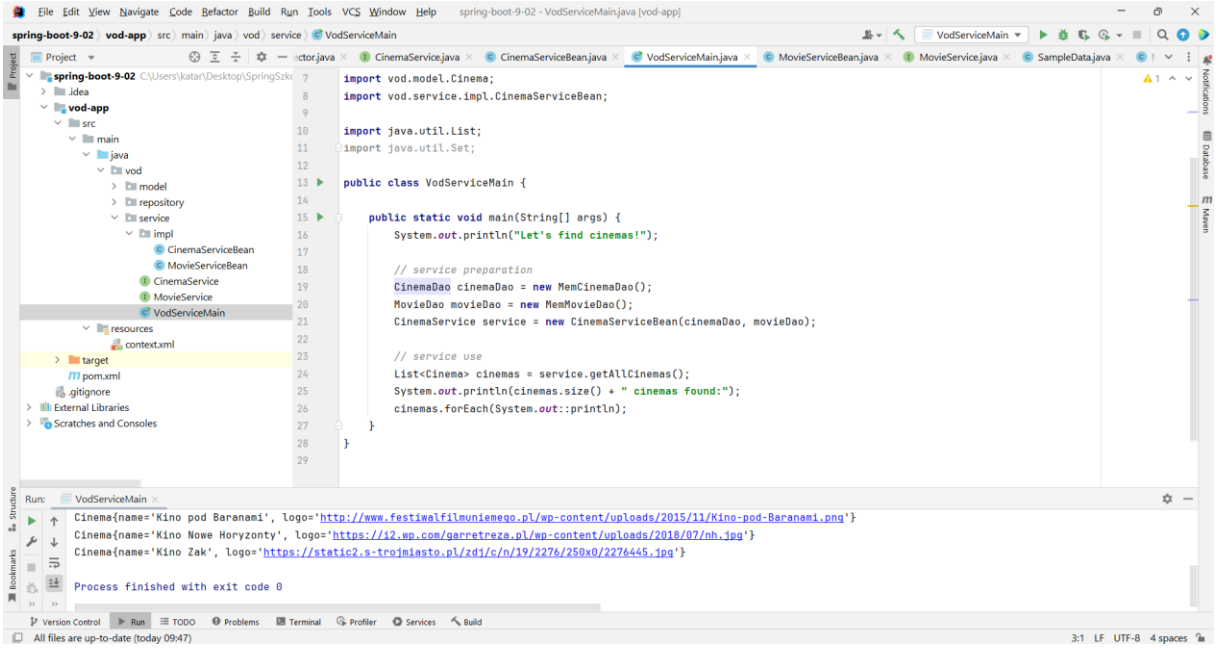
Obiekt Logger służy do rejestrowania komunikatów dla określonego systemu lub komponentu aplikacji.

Warstwa biznesowa nie skupia się na tym jak będzie dostępna, czy kanałem Restowym czy Web serwisy SOAPowe, czy EJB – klient zdalnego dostęp, czy UI, czy stand alone.

To z punktu widzenia domeny jest nieistotne bo to są funkcjonalności wystawione przy pomocy API Javowego.

Klasa startowa pozwoli nam poznać podstawy działania kontenera zależności, później przełączymy się na starter docelowy springbootowy.

Użycie jednego serwisu CinemaServiceBean.



Po uruchomieniu programu z punktu widzenia klienta aplikacji, którym jest klasa główna czyli mainowa wszystko jest ok, klasa z main, która jest klientem serwisu musi sobie sama ten serwis zapewnić. Tego chcemy nie robić bo potrzebujemy cinemaService ale nie chcemy go przygotowywać. Jak się nazywa klasa, jak wygląda konstruktor, jakie przyjmuje typu, jak to interfejsowe typy to skąd wziąć implementacje, które wybrać to jest za dużo szczegółów z punktu widzenia klienta, który chce skorzystać z już skonfigurowano sewisu.

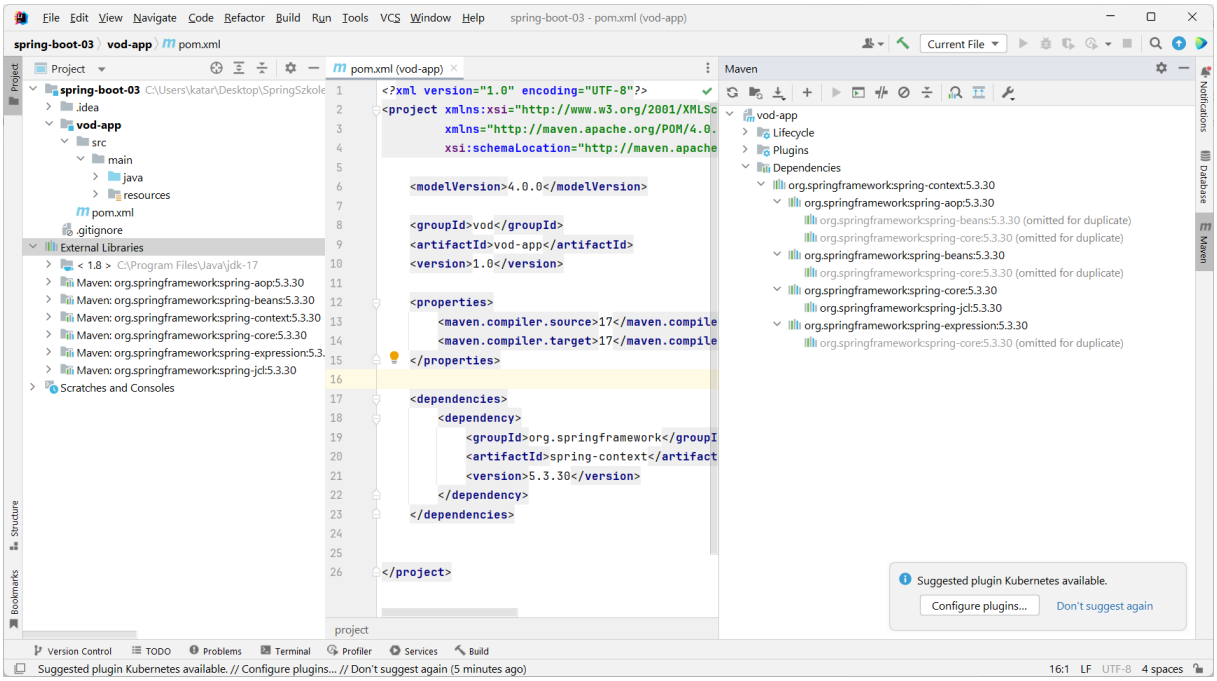
Spróbujemy teraz to osiągnąć korzystając jeszcze nie ze springboota ale w pierwszym kroku bezpośrednio z zależności do spring kontekstu.

W pom.xml (project object model) dodaj grupę zależności a w niej jedną zależność (dependency)

```
<dependencies>
    <dependency>
        <groupId>org.springframework</groupId> //grupa
        <artifactId>spring-context</artifactId> //artefakt
Zawiera w sobie kontener
        <version>5.3.30</version>
    </dependency>
</dependencies>
```

Odśwież mavena np. „latającym guzikiem” reload lub w sekcji mavena jest guzik do przeładowania.

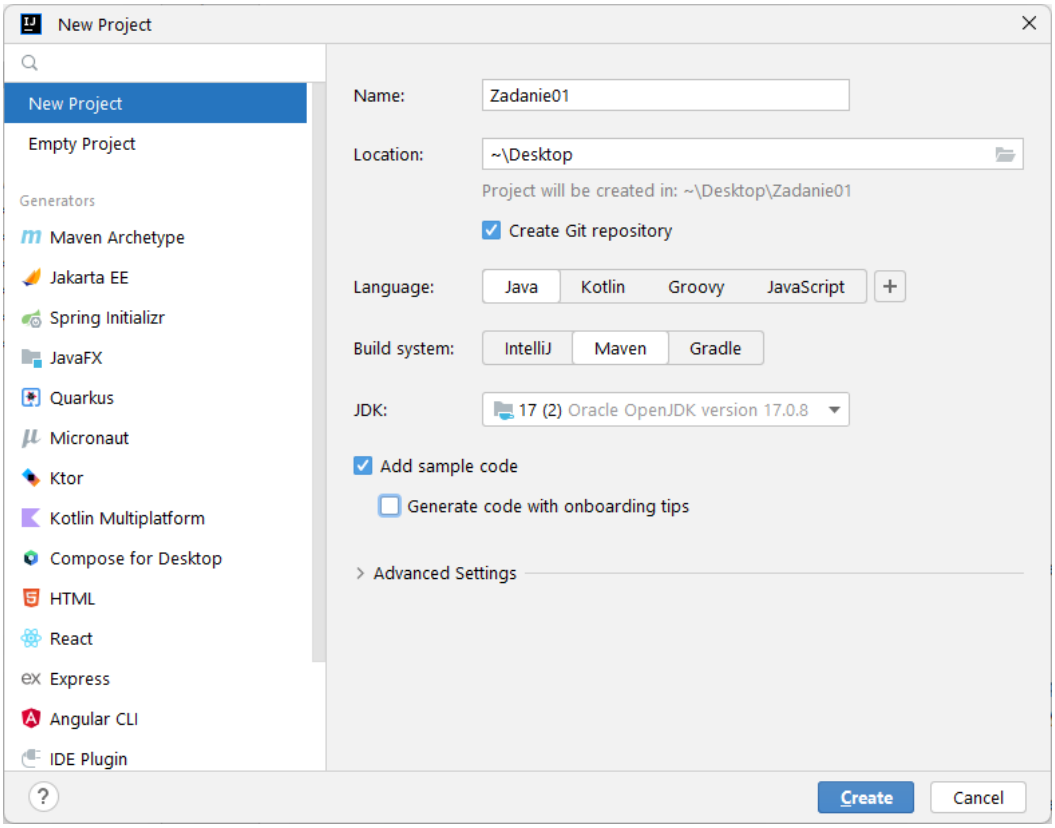
W external libraries pojawiły się biblioteki z których będziemy korzystali.



Na razie skupiamy się na bootstrapie aplikacji (potem dodamy spring bota i aspekt webowego).

Zadanie01

Utwórz podobną aplikację ze swoją własną logiką biznesową. Ustaw.



Po utworzeniu dostosuj strukturę do otrzymanej - **spring-boot-02 ze swoją własną logiką biznesową.**

Przeanalizuj kod każdej klasy.